

但如何知道注入服務會是何種實例？可檢視 Program.cs 中服務註冊 IBankService 介面對應的實作類別為 FubonService：



Program.cs

```
//將網銀服務註冊到 DI Container
builder.Services.AddTransient<IBankService, FubonBankService>();
//builder.Services.AddTransient<IBankService, EsunBankService>();
```

介面

類別實作

以下是 ASP.NET Core 使用相依性注入步驟：

1. 建立 interface 介面，宣告屬性與方法
2. 建立 Service 服務類別，繼承 interface 介面並實作
3. 在 Program.cs 中註冊 Service 服務
4. 在 Controller、Action 或 View 中注入 Service 相依性物件，然後使用

範例 13-1 網銀服務之 DI 相依性注入建立與調用

以下示範網銀服務相依性注入之運用，從介面宣告、實作服務、在 DI Container 註冊服務，最後於 Controller 建構函式以 DI 注入網銀服務。

step 01 建立網銀服務 interface 介面，宣告兩個屬性及三個方法



Interfaces/IBankService.cs

```
public interface IBankService
{
    string BankId { get; }
    string BankName { get; }
    //查詢帳戶餘額
    decimal AccountBalance(int depositorId);
    //提款
    bool Withdraw(decimal dollars);
    //存款
    bool Deposit(decimal dollars);
}
```

step 02 建立 FubonBankService 服務(台北富邦銀行), 繼承 IBankService 介面, 實作屬性與方法, 而方法僅象徵性做簡單回傳值

 Services/FubonBankService.cs

```
public class FubonBankService : IBankService ← 繼承共通介面
{
    public string BankId { get; private set; }
    public string BankName { get; private set; }

    public FubonBankService()
    {
        BankId = "012";
        BankName = "台北富邦銀行";
    }

    public decimal AccountBalance(string depositorId)
    {
        decimal balance = 1000000;
        if (depositorId == "18072")
        {
            balance = 5000000;
        }
        return balance;
    }

    public bool Deposit(decimal dollars)
    {
        //Todo ...
        return true;
    }

    public bool Withdraw(decimal dollars)
    {
        //Todo ...
        return true;
    }
}
```

step 03 建立 ESunBankService 服務（玉山銀行），繼承 IBankService 介面，實作屬性與方法



Services/ESunBankService.cs

```
public class EsunBankService : IBankService ← 繼承共通介面
{
    public string BankId { get; private set; }
    public string BankName { get; private set; }

    public EsunBankService()
    {
        BankId = "808";
        BankName = "玉山銀行";
    }

    public decimal AccountBalance(string depositorId)
    {
        decimal balance = 3000000;
        if (depositorId == "18072")
        {
            balance = 1500000;
        }
        return balance;
    }

    public bool Deposit(decimal dollars)
    {
        //Todo ...
        return true;
    }

    public bool Withdraw(decimal dollars)
    {
        //Todo ...
        return true;
    }
}
```

step 04 在 Program.cs 中註冊 FubonBankService 服務

在網銀服務能夠以 DI 注入前，須先在 Program.cs 中註冊網銀服務相依性，那麼 FubonBankService 與 ESunBankService 服務由於是對等的實作，一般一次僅會使用其中一個，例如起初使用的是 FubonBankService 服務，註冊方式如下：

 Program.cs

```
//將網銀服務註冊到 DI Container
builder.Services.AddTransient<IBankService, FubonBankService>();
```

介面 台北富邦銀行實作

說明：在 DI Container 中註冊服務有 AddTransient、AddScoped 和 AddSingleton 三種方法，代表服務三種生命週期，後續會再解釋

step 05 在 Bank 控制器的建構函式以 DI 注入網 IBankService，並在 Balance()方法中使用該服務 Conrollers/BankController.cs

```
namespace Mvc7_DependencyInjection.Controllers
{
    public class BankController : Controller
    {
        private readonly IBankService _bankService;

        public BankController(IBankService bankService)
        {
            _bankService = bankService;
        }

        public IActionResult Balance()
        {
            ViewData["BankId"] = _bankService.BankId;
            ViewData["BankName"] = _bankService.BankName;
            ViewData["Balance"] = _bankService
                .AccountBalance("18072").ToString("C");

            return View();
        }
    }
}
```

相依性注入標準樣板

Constructor 建構函式注入

注入相依性物件實例

將相依性物件指派給 _bankService

```
}  
}
```

說明：使用相依性注入不需手動初始化類別實例，只需在類別建構函式以參數形式宣告要注入的服務，型別為 `IBankService` 抽象介面，便能獲得網銀服務實例

step06 在 View 顯示銀行帳戶餘額

 Views/Bank/Balance.cshtml

```
...  
<ul>  
  <li>銀行代碼 : @ViewData["BankId"]</li>  
  <li>銀行名稱 : @ViewData["BankName"]</li>  
  <li>存款餘額 : @ViewData["Balance"]</li>  
</ul>
```

瀏覽 Bank/Balance 網址，會看見顯示的是「台北富邦銀行」帳戶餘額，表示系統注入的是 `FubonBankService` 物件。

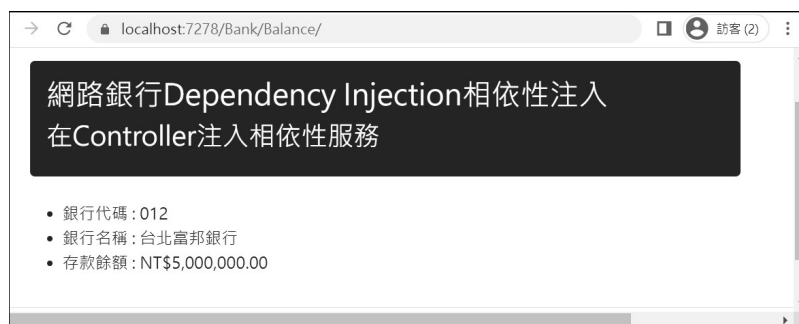


圖 13-1 注入 `FubonBankService` 服務實例

step07 假設日後欲將服務實作改成玉山銀行，只需在 DI Container 中替換成 `ESunBankService`

 Program.cs

```
builder.Services.AddTransient<IBankService, ESunBankService>();
```

介面

玉山網銀實作

重新執行，瀏覽 Bank/Balance 網址，便會顯示「玉山銀行」帳戶餘額，表示系統注入的是 ESunBankService 物件。

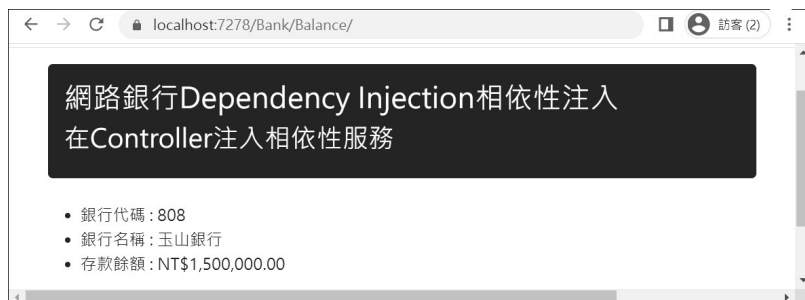


圖 13-2 注入 ESunBankService 服務實例

由上可知，透過 DI 方式實作和調用服務，除了可達成鬆散耦合設計，且在抽換類別服務實作時，達到無痛移轉，或將衝擊影響最小化。

13-3-2 在 Action 使用相依性注入

一般在 Controller 建構函式調用 DI，共用性較佳，因為可讓所有的 Actions 共用 DI 物件。但也可在個別 Action 方法中使用 DI，那何時會使用此方式？就是只有該 Action 會用到相依性物件，方式是在 Action 的參數型用[FromServices]宣告網銀服務注入：

Controllers/BankController.cs

```
public IActionResult InjectAction([FromServices] IBankService _bankService)
{
    ViewData["BankId"] = _bankService.BankId;
    ViewData["BankName"] = _bankService.BankName;
    ViewData["Balance"] = _bankService.AccountBalance("18072").ToString("C");

    return View();
}
```

在 Action 中注入相依性物件